CURSO:
ESTRUCTURA DE DATOS CON POO

Docente: M(o). Ing. Loncán Salazar, Pierre Paul

SESIÓN Nº 04

AGENDA:

- **Algoritmos de Recursivo**
  - QuickSort
  - MergeSort

RECURSIVIDAD

M(o). Ing. Loncán Salazar, Pierre Paul

3

RECURSIÓN

- La recursión es un enfoque orientado a la resolución de problemas, basado en la división y conquista (o división y combinación):

```
método Recursivo(Argumentos)
{
  Si (Argumentos suficientemente Pequeños)  // "Terminación"
    retorna Respuesta
  sino                                      // "Dividir"
    Identidad = Combinar(Recursivo(Argumentos más pequeños),
                         Recursivo(Argumentos más pequeños))
    retorna Identidad                       // "Combinar"
  Fin Si
}
```

- Si puede escribir un problema como si fuese la combinación de pequeños problemas, puede implementarlo a través de un algoritmo recursivo de Java.

M(o). Ing. Loncán Salazar, Pierre Paul

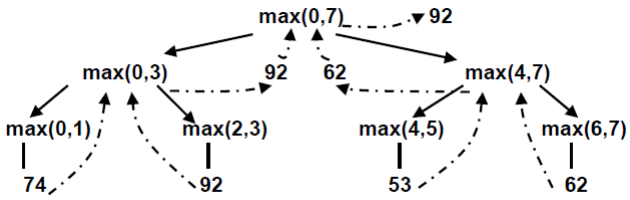
4

HALLANDO EL VALOR MÁXIMO DE UN ARRAY

- Suponga que sólo podemos hallar el valor máximo de 2 números cada vez.
- Imagine que queremos averiguar el valor máximo de un conjunto de 8 números.

```
35   74   32   92        53   28   50   62
```

- El método recursivo **`max`** se llama a sí mismo:



M(o). Ing. Loncán Salazar, Pierre Paul

CÓDIGO PARA EL MÉTODO MAX

```
int[] AData= {35, 74, 32, 92, 53, 28, 50, 62};
System.out.println("Max: " + maxArray(0, 7, AData));
```

```
public int combinar(int a, int b)
{
   return ( (a >= b) ? a : b);
}
public int maxArray( int i, int j, int[] Arr)
{
   if ( (j - i) <= 1)     // Suficientemente pequeño
   {
      if (Arr[j] >= Arr[i])
         return Arr[j];
      else
         return Arr[i];
   }
   else                   //Dividir y combinar
      return (combinar(maxArray(i, (i+j)/2, Arr),
                       maxArray((i+j)/2+1, j, Arr)));
}
```

M(o). Ing. Loncán Salazar, Pierre Paul

EXPONENCIACIÓN

- La exponenciación, hecha "de forma simple", no es eficaz.
  - Elevar **x** a la potencia de **y** puede admitir **y** multiplicaciones:
    - Ejemplo: $x^7 = x * x * x * x * x * x * x$
  - Calcular el cuadrado consecutivo es mucho más eficiente, pero hay que ser cauteloso en su implementación.
    - Por ejemplo: $x^{48} = ((((x * x * x)^2)^2)^2)^2$ utiliza 6 multiplicaciones en vez de 48.
- Informalmente, la exponenciación simple O(n).
  - El cuadrado es O(lg n), dado que elevar un número a la enésima potencia conlleva lg n operaciones (base 2).
    - **Lg(48)= Log2(48)= sobre 6**
    - **232 = 5; 264 = 6**
  - **Para** hallar $x^{1,000,000,000}$, calcular el cuadrado conlleva **30** operaciones, mientras que con el método tradicional serían 1,000,000,000

M(o). Ing. Loncán Salazar, Pierre Paul

7

EXPONENCIACIÓN RECURSIVA

- Pseudocódigo:

```
metodo ExpResult(x, y)
{
   Si (y == 1)              // y es suficientemente pequeño
      Resultado = x
   Sino si (y mod 2 == 0)   // y par
      Resultado =  square(ExpResult(x, y/2))
Sino                        // y impar
      Resultado = x * ExpResult(x, y-1)
   Retornar Resultado
}
```

- Una identidad (condición) devuelve una respuesta real.
  - Un algoritmo recursivo debe poseer un caso lo bastante pequeño como para terminar la recursión al final.
- Dos de las identidades (condiciones) disminuyen el problema en cada paso.
  - Un algoritmo recursivo ha de reducir el problema en cada paso.

M(o). Ing. Loncán Salazar, Pierre Paul

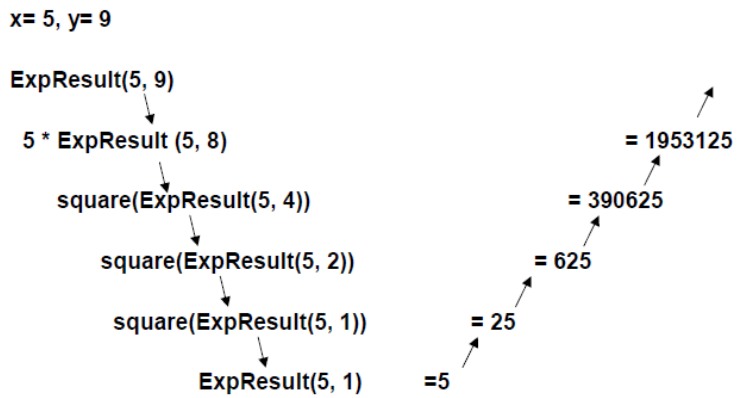
8

Exponenciación Recursiva

- Tres identidades:
  - $x^1 = x$ (lo bastante pequeño)
  - $x^2 = x * x$ (reduce el problema) → Exponente Par
  - $x^3 = x * x^2$ (reduce el problema) → Exponente Impar

M(o). Ing. Loncán Salazar, Pierre Paul

Cómo funciona la recursión



M(o). Ing. Loncán Salazar, Pierre Paul

CÓDIGO DE EXPONENCIACIÓN

```
public static int ExpResult(int x, int y)
{
   int resultado;
   if (y == 1)                          // y lo bastante pequeño
      resultado = x;
   else if (y % 2 == 0)                 // y par
   {
      int term = ExpResult(x, y/2);
      resultado = term * term;          // cuadrado(ExpResult(x, y/2)
   }
   else                                 // y impar
      resultado = x * ExpResult(x, y-1);
      return resultado;
}

// Utilice BigInteger para manejar números mayores
```

M(o). Ing. Loncán Salazar, Pierre Paul

11

RECURSIÓN E ITERACIÓN

- Escribir la exponenciación iterativamente es una tarea difícil.
  - Inténtelo si está interesado y dispone de tiempo.
- Normalmente, es más fácil ver una implementación recursiva correcta.
  - La recursión con frecuencia se acerca más a la matemática subyacente.
- Existe un modo mecánico, empleado por compiladores y diseñadores de algoritmos, encargado de convertir la recursión en iteración. Es complejo y se utiliza para mejorar la eficiencia.
  - El consumo de las llamadas a métodos es evidente, y la conversión de recursión a iteración dentro de un método acelera la ejecución.
  - Los métodos pequeños o utilizados con poca frecuencia pueden quedar como recursivos.

M(o). Ing. Loncán Salazar, Pierre Paul

12

Ordenación Recursiva

M(o). Ing. Loncán Salazar, Pierre Paul

Ordenación Recursiva - MergeSort (Método MergeSort)

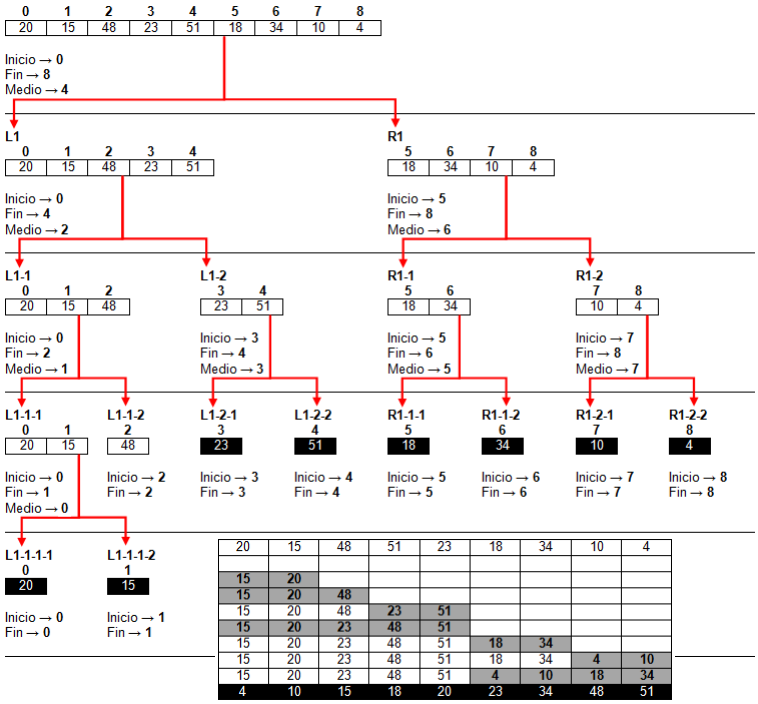
- La ordenación por fusión es un método de ordenación recursivo que utiliza el método de fusión para realizar la ordenación completa de un array.

```
public void mergesort(int[ ] Datos, int Inicio, int Fin)
{
  if (Inicio < Fin)
  {
     int Medio = (Inicio + Fin) / 2;    // Posición central del total de
 elementos

     mergesort(Datos, Inicio, Medio);        // Ordenar lado izquierdo
     mergesort(Datos, Medio + 1 , Fin);      // Ordenar lado derecho

     merge(Datos, Inicio, Medio, Fin);       // Fusionar ambos lados
  }
}
```

M(o). Ing. Loncán Salazar, Pierre Paul



ORDENACIÓN RECURSIVA - MERGESORT (MÉTODO DE FUSIÓN MERGE)

```
public void Merge(int Datos[],int Inicio, int Medio, int Fin)
{
    int n1 = Medio-Inicio+1;            //N° de elementos del Arreglo Izquierdo
    int n2 = Fin-Medio;                 //N° de elementos del Arreglo Derecho
    int Izquierda[] = new int[n1+1];    // Crea el arreglo Izquierda
    int Derecha[] = new int[n2+1];      // Crea el arreglo Derecha
    for(int i = 0; i < n1; i++)
    {
        Izquierda[i] = Datos[Inicio+i]; //Copia el contenido en Arreglo Izquierda
    }
    for(int j = 0; j < n2; j++)
    {
        Derecha[j] = Datos[Medio+j+1];  //Copia el contenido en Arreglo Derecha
    }
    Izquierda[n1] = Integer.MAX_VALUE;  // Añade al arreglo el máximo valor entero
    Derecha[n2] = Integer.MAX_VALUE;    // Añade al arreglo el máximo valor entero
    int i = 0, j = 0;
    for(int k = Inicio; k <= Fin; k++)
    {
        if(Izquierda[i] <Derecha[j])
        {
            Datos[k] = Izquierda[i];    //Copia valor si izquierda es menor
            i++;
        }
        else
        {
            Datos[k] = Derecha[j];      //Copia valor si derecha es menor
            j++;
        }
    }
}
```

ORDENACIÓN RECURSIVA - QUICKSORT (MÉTODO QUICKSORT)

- Quicksort reúne lo mejor de todos los métodos de ordenación vistos hasta ahora, su base está en el particionado el cual conseguirá un orden de logaritmo en base 2.

```
public void Quicksort(int[] Datos, int Primero, int Ultimo)
    {
        int pivote;
        if(Primero < Ultimo)
        {
          pivote = Colocar(Datos, Primero, Ultimo);
            Quicksort(Datos, Primero, pivote-1);
            Quicksort(Datos, pivote+1, Ultimo);
        }
    }
```

M(o). Ing. Loncán Salazar, Pierre Paul

17

ORDENACIÓN RECURSIVA - QUICKSORT (MÉTODO COLOCAR)

```
public int Colocar(int[] Datos, int Primero, int Ultimo)
    {
       int i;
       int pivote, valor_pivote;
       int temp;

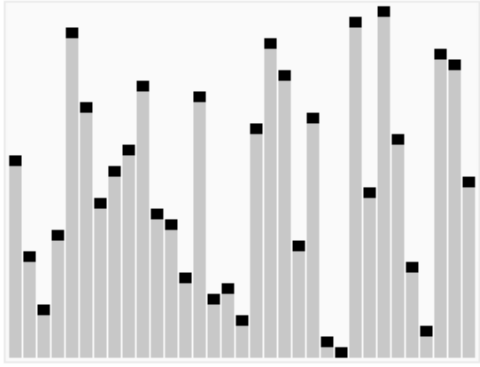
       pivote = Primero;                //Establecer la Posición del Pivote
       valor_pivote = Datos[pivote];    //Extraer el valor del Pivote
       for (i = Primero+1; i<=Ultimo; i++)
       {
          if ( Datos[i] < valor_pivote ) //Hallar y Transponer los Valores
          {                               //menores al Pivote
             pivote++;
             temp = Datos[i];
             Datos[i] = Datos[pivote];
             Datos[pivote] = temp;
          }
       }

       //Colocar el Pivote en su Posición Final
       temp = Datos[Primero];
       Datos[Primero] = Datos[pivote];
       Datos[pivote] = temp;
       return pivote;
    }
```

M(o). Ing. Loncán Salazar, Pierre Paul

18

Ordenación Recursiva - Quicksort (Video - Demostración)



M(o). Ing. Loncán Salazar, Pierre Paul

19



M(o). Ing. Loncán Salazar, Pierre Paul

20